

Talents Connect

Application Next.js full-stack (App Router) — un seul projet pour le frontend, les server actions et l'accès aux données, connecté à une base PostgreSQL via Prisma.

Next.js 16 (App Router)

TypeScript

PostgreSQL

Prisma 7

NextAuth (Auth.js) v5

Tailwind CSS v4

Zod

otplib (TOTP)

pdfkit

Resend

01 Stack technique

COUCHE	CHOIX	RÔLE
Framework	Next.js 16, App Router	Rendu serveur (RSC), routage par dossier, server actions
Langage	TypeScript	Typage de bout en bout, y compris le client Prisma généré
Base de données	PostgreSQL	Stockage relationnel de l'ensemble des données
ORM	Prisma 7 (driver adapter <code>@prisma/adapters-pg</code>)	Schéma, migrations, requêtes typées
Authentification	NextAuth (Auth.js) v5, stratégie JWT	Comptes Credentials + OAuth (Google/LinkedIn) + 2FA cabinet
Style	Tailwind CSS v4 (jetons CSS)	Design system de la marque Talents Consult
Validation	Zod	Validation des formulaires côté serveur
Mots de passe	bcryptjs	Hachage des mots de passe Credentials et des codes de secours 2FA
Double authentification	otplib v12 (TOTP) + qrcode	Génération/validation des codes 2FA et du QR code d'enrôlement
Reçus PDF	pdfkit	Génération à la volée des reçus de paiement
E-mail transactionnel	Resend	Envoi réel des notifications, vérifications et liens de sécurité

02 Structure du projet

Les routes sont organisées par façade — chaque groupe de routes correspond à un espace applicatif protégé par rôle.

```
# extrait – arborescence simplifiée
prisma/
  schema.prisma      # ~35 modèles, migrations versionnées
  seed.ts            # compte admin cabinet + offres de démo
src/
  proxy.ts           # middleware – protège /client, /cabinet, /candidate
  app/
    (public-jobs)/    # job board public : /, /results, /jobs/[id], /companies/[id]...
    (auth)/           # /login, /register, /forgot-password, /reset-password,
  /verify-email...
    api/invoices/[id]/ # route handler : stream du reçu PDF, contrôle d'accès
    client/           # espace Client (protégé, rôle CLIENT) – dont /cv-database,
  /settings
    cabinet/          # espace Cabinet (protégé, rôle CABINET) – dont /prospects,
  /admin/kpi
    candidate/        # espace Candidat (protégé, rôle CANDIDATE)
  components/
    ui/              # Button, Card, Tag, Field, Tabs – primitives partagées
    mandats/ client/ cabinet/ candidate/ jobs/ training/ layout/ account/ companies/
prospects/
  lib/
    auth.ts          # config NextAuth complète (Node) – providers, callbacks, 2FA
    auth.config.ts   # config partagée, compatible Edge (utilisée par proxy.ts)
    prisma.ts        # instance PrismaClient singleton
    rbac.ts          # requireRole() / requireAdmin() / getSessionUser() /
getEmailVerified()
  storage.ts         # upload de fichiers (CV) – validation magic-bytes,
adaptateur disque local
  rateLimit.ts       # limitation de fréquence par clé (connexion, inscription,
candidature...)
  mfa.ts             # génération/validation TOTP + codes de secours
  invoice.ts         # numérotation séquentielle + génération PDF des reçus
  email.ts, notify.ts # envoi Resend + écriture Notification
  validations/       # schémas Zod par domaine
server/
  actions/           # logique métier ("use server") par domaine
  services/
    mandatStateMachine.ts # transitions de statut du mandat
    prospectStateMachine.ts # transitions du pipeline de prospection commerciale
```

03 Modèle de données

Une trentaine de modèles Prisma, organisés en domaines. Le détail des relations et un schéma visuel sont dans le document Architecture.

Identité & comptes

User , Company , ClientProfile , CabinetProfile , CabinetInvite , Candidate

Mandats

Mandat , MandatStatusEvent , MandatMessage , MandateApplication

Entretiens

Interview , InterviewSlot

Job board

JobPosting , JobApplication , DirectMessage , JobSponsorship , JobAlert

Formations

TrainingCourse , TrainingSession , Enrollment

CVthèque & confiance

CvDatabaseAccess , CompanyReview

Facturation

Invoice (numérotation par année, source polymorphe : sponsoring / CVthèque / formation)

Prospection commerciale

Prospect , ProspectStageEvent

Sécurité & comptes

EmailVerificationToken , PasswordResetToken , RateLimitAttempt

Transverse

Notification

Le rôle utilisateur (Role : CLIENT / CABINET / CANDIDATE) porte à lui seul l'aiguillage vers la bonne façade — aucun rôle Admin séparé : les droits d'administration cabinet sont un simple booléen isAdmin sur CabinetProfile , qui porte aussi les champs de la double authentification (totpSecret , totpEnabled , totpBackupCodes).

04 Authentification & autorisation

NextAuth est scindé en deux fichiers pour respecter la contrainte du middleware Next.js, qui s'exécute en *Edge Runtime* (aucun accès à Prisma/Node) :

- lib/auth.config.ts — callbacks jwt / session uniquement, aucun accès base de données. Utilisé directement par proxy.ts .
- lib/auth.ts — étend la config partagée avec les providers réels (Credentials, Google, LinkedIn conditionnels) et un callback signIn qui interroge Prisma pour créer/retrouver l'utilisateur.

Protection des routes

proxy.ts (middleware) redirige vers /login tout accès non authentifié à /client/* , /cabinet/* ou /candidate/* . Chaque layout de façade revérifie ensuite le rôle côté serveur via requireRole() , et chaque server action revérifie indépendamment rôle et propriété des données — la protection ne repose jamais sur un seul point de contrôle.

Connexion sociale

Google et LinkedIn sont ajoutés dynamiquement aux providers uniquement si leurs variables d'environnement sont définies (googleEnabled / linkedinEnabled , exportées pour l'UI). Réserve au rôle Candidat : le callback signIn crée le User + Candidate au premier passage (avec emailVerified déjà renseigné, l'adresse étant déjà vérifiée par le fournisseur), et refuse la connexion si l'e-mail correspond déjà à un compte non-candidat.

Double authentification (comptes cabinet)

Le provider Credentials accepte un champ totpCode optionnel. Si CabinetProfile.totpEnabled est vrai, authorize() exige un code TOTP valide (fenêtre glissante de 30 s) ou l'un des codes de secours à usage unique (hachés en base, consommés à l'usage) avant de retourner une session. Côté client, le formulaire de connexion interroge d'abord checkRequiresMfa(email) — sur la même clé de limitation de fréquence que la connexion elle-même — pour savoir s'il doit révéler le second champ, sans jamais court-circuiter la vérification serveur.

05 Server actions & sécurité

Toute la logique métier vit dans `src/server/actions/*.ts`, sous forme de fonctions `"use server"` appelées directement depuis les composants client — pas de couche API REST intermédiaire (seule exception : `api/invoices/[id]/route.ts`, un vrai route handler, nécessaire pour streamer un fichier binaire avec ses en-têtes). Chaque action suit le même schéma :

1. `requireRole("CLIENT")` // rejette si non connecté / mauvais rôle
2. Validation Zod des données reçues
3. Vérification de propriété // ex: `mandat.companyId === user.companyId`
4. Écriture Prisma (transaction si plusieurs tables)
5. `revalidatePath()` des pages concernées

Les transitions de statut de mandat passent systématiquement par `mandatStateMachine.ts`, qui valide la transition demandée contre une table de transitions autorisées et journalise chaque changement dans `MandatStatusEvent` — même principe pour `prospectStateMachine.ts` et `ProspectStageEvent` côté prospection commerciale. Aucun code applicatif n'écrit `Mandat.status` ou `Prospect.stage` directement.

Piège Next.js identifié en cours de développement

Après toute server action, Next.js re-rend automatiquement les Server Components de la route courante — y compris sans appel explicite à `revalidatePath()` ni `router.refresh()`. Un composant dont l'affichage dépend d'une condition calculée côté serveur (ex. "avis déjà soumis ?") peut donc se démonter avant que l'utilisateur voie la confirmation. Le correctif retenu : garder l'état de confirmation dans un composant client stable, en position fixe dans l'arbre, qui prend le pas sur les props recalculées côté serveur.

06 Modules ajoutés (v2)

Ajoutés après la première version fonctionnelle, en réponse à un audit des manques pour un usage réel.

Limitation de fréquence (`lib/rateLimit.ts`)

Implémentation en base (table `RateLimitAttempt`), sans dépendance externe type Redis — une clé libre (`login:{email}` , `register:{ip}` , `apply:{candidateId}` ...) et une fenêtre glissante en minutes. Suffisant au volume actuel ; à revoir si le trafic devait fortement augmenter.

Validation des fichiers uploadés (`lib/storage.ts`)

Vérification de la signature binaire (magic bytes) du fichier reçu contre une liste blanche PDF/DOC/DOCX, en plus de sa taille — l'extension et le type déclarés par le navigateur ne sont jamais utilisés seuls.

Facturation (`lib/invoice.ts`)

Un reçu (`Invoice`) est créé dans la même transaction que la confirmation de paiement (sponsoring, CVthèque, formation), avec un numéro séquentiel par année (`INV-{année}-{0000}`), calculé par comptage — adapté au faible volume de confirmations manuelles, pas conçu pour tenir sous forte concurrence). Le PDF est généré à la demande par `generateInvoicePdf()` (pdfkit) et servi par `api/invoices/[invoiceId]/route.ts` , qui vérifie que l'appelant est soit le titulaire du reçu, soit un compte cabinet.

Droit à l'oubli (`server/actions/accountDeletion.ts`)

Le compte `User` n'est jamais supprimé en base — il est référencé comme auteur/acteur dans de nombreuses tables (mandats, messages, avis, offres) à des fins d'historique. La suppression anonymise à la place les champs personnels (nom, e-mail, mot de passe rendu inutilisable) et supprime les données strictement personnelles (CV, alertes, notifications, jetons). Bloquée si le compte cabinet est le seul administrateur, pour éviter un verrouillage total de l'administration.

Statistiques globales (`server/actions/kpis.ts`)

Agrégations Prisma (`groupBy` , `count` , `aggregate`) exécutées en parallèle (`Promise.all`) : comptes par rôle, offres, candidatures et taux de recrutement, mandats par statut et taux de réussite, entretiens, revenu total et par source.

07 Variables d'environnement

VARIABLE	REQUISE	RÔLE
DATABASE_URL	Oui	Connexion PostgreSQL
AUTH_SECRET	Oui	Secret de signature des sessions NextAuth
NEXTAUTH_URL	Oui	URL publique de l'application (utilisée dans les liens des e-mails)
GOOGLE_CLIENT_ID / _SECRET	Non	Active la connexion Google si définies
LINKEDIN_CLIENT_ID / _SECRET	Non	Active la connexion LinkedIn si définies
RESEND_API_KEY	Non	Active l'envoi réel d'e-mails ; sans elle, les e-mails sont journalisés côté serveur au lieu d'être envoyés
EMAIL_FROM	Non	Adresse d'expédition des e-mails (par défaut le domaine de test Resend)
SEED_CABINET_ADMIN_EMAIL / _PASSWORD	Non	Surcharge les identifiants du compte admin créé par db:seed

08 Scripts npm

COMMANDE	EFFET
npm run dev	Serveur de développement (Turbopack)
npm run build	Build de production
npm run db:migrate	prisma migrate dev — applique le schéma en local
npm run db:seed	Crée le compte admin cabinet + les offres de démonstration
npm run db:studio	Ouvre Prisma Studio (exploration de la base)
npm run vercel-build	prisma migrate deploy && next build — build de déploiement en une étape

09 Vérification & tests

Aucune suite de tests automatisés n'a été mise en place ; la vérification a été faite en continu, à chaque fonctionnalité :

- `npm run build` et `npx tsc --noEmit` systématiques avant chaque livraison.
- `npm run lint` (ESLint, config Next.js) systématique.
- Parcours de bout en bout dans un navigateur réel (Playwright headless) pour chaque fonctionnalité livrée, y compris les scénarios de sécurité : validation de CV falsifiés, dépassement de la limitation de fréquence, cycle complet d'enrôlement/connexion/désactivation de la 2FA (avec génération de codes TOTP réels), téléchargement d'un reçu et contrôle d'accès (un tiers ne peut pas le récupérer), suppression de compte suivie d'un test de connexion refusée.

Recommandation

Ajouter une suite de tests automatisés (Vitest/Playwright) avant toute mise en production réelle — la couverture actuelle repose sur des vérifications manuelles répétées à chaque livraison, pas sur des tests exécutables en continu (CI).

10 Déploiement

1. Créer une base PostgreSQL managée (Neon ou Supabase, offre gratuite suffisante pour démarrer).
2. Importer le dépôt GitHub dans un nouveau projet Vercel.
3. Renseigner les variables d'environnement de la section 7 dans les réglages du projet Vercel (au minimum les trois obligatoires ; `RESEND_API_KEY` fortement recommandée pour que les e-mails de sécurité partent réellement).
4. Vercel détecte automatiquement `vercel-build` et l'exécute — migrations puis build, en une étape.

Détail pas-à-pas (y compris l'obtention des identifiants Google/LinkedIn/Resend) dans `README.md` à la racine du dépôt. L'application n'a pas encore été déployée en production à ce jour — elle a été développée et testée dans un environnement de développement local.

11 Dette technique connue

SUJET	ÉTAT	RECOMMANDATION
Tests automatisés	Aucun	Ajouter Vitest (unitaire) + Playwright (bout en bout) et les brancher en CI
Stockage de fichiers	Disque local (<code>lib/storage.ts</code>)	Basculer vers un stockage objet (S3-compatible) avant mise à l'échelle — un disque local ne survit pas à un redéploiement sur la plupart des plateformes
Paie­ments	Suivi manuel (formations, sponsoring, CVthèque), reçu généré automatiquement	Intégrer une passerelle une fois un compte marchand disponible (Stripe ou CMI selon les marchés visés)
Recherche	Filtrage SQL simple (offres, candidats)	Suffisant au volume actuel ; revoir (Elasticsearch/OpenSearch) si le catalogue grossit fortement
Numérotation des reçus	Comptage simple par année, sans verrou dédié	Passer à un compteur atomique si le volume de confirmations manuelles augmente fortement
Mise en production	Jamais déployée en ligne	Suivre la procédure de la section 10 ; valider d'abord sur un environnement de staging